TP 1 — Faire parler une API que personne ne documente

Brancher un serveur MCP inconnu, puis transformer ce que votre agent a appris en un skill réutilisable

OBJECTIFS PÉDAGOGIQUES

- Brancher un serveur MCP dans un agent de code et vérifier qu'il répond vraiment.
- Piloter un agent sur une API dont il n'existe aucune documentation, ni en ligne ni dans son entraînement.
- Reconnaître qu'un agent qui « a réussi » peut avoir travaillé sur des données fausses.
- Capitaliser : transformer une session de galère en un *skill* qui évite la galère suivante.

CONTEXTE

La médiathèque municipale a fait développer son logiciel de gestion il y a douze ans, par un prestataire qui n'existe plus. Il reste une API, un serveur MCP posé par-dessus, et **zéro documentation** : la personne qui savait s'en servir est partie à la retraite.

On vous confie cinq tâches sur ce système. Vous n'aurez pas le temps de les faire à la main : c'est votre agent qui travaille. Votre travail à vous, c'est de comprendre l'API mieux que lui, et de l'écrire.

PRÉREQUIS & MISE EN PLACE

Vous recevez une **URL** et un **token personnel** en début de séance. Le token vous identifie côté serveur : ne le partagez pas, vous partageriez aussi votre quota. Tout le TP se fait sous **OpenCode**.

```
$ mkdir tp1-<nom1>-<nom2> && cd tp1-<nom1>-<nom2>
$ git init
```

DÉCLARER LE SERVEUR MCP

Dans `opencode.json`, à la racine de votre projet :

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "bibliotheque": {
      "type": "remote",
      "url": "<URL fournie en séance>",
      "enabled": true,
      "headers": { "Authorization": "Bearer <votre-token>" }
    }
  }
}
```

Ce fichier de configuration fait partie du rendu. **Le token, lui, n'est jamais commité** : sortez-le dans une variable d'environnement ou un fichier ignoré par Git, et dites dans votre rapport comment vous vous y êtes pris.

1 Intégrer le serveur MCP

Objectif : prouver que la liaison fonctionne de bout en bout. Tant que cette partie n'est pas acquise, la suite est impossible.

a. Configurez le serveur MCP dans OpenCode et redémarrez-le. Faites-lui **lister les outils exposés** par le serveur. Capture d'écran.

b. Faites-lui appeler **un** outil de lecture et capturez la réponse brute, celle qui contient de vraies données de la médiathèque — pas le résumé qu'en fait l'agent.

c. Le serveur expose des outils qui décrivent vos cinq missions. Trouvez-les, et recopiez dans votre rapport **l'énoncé exact des cinq missions**. Elles ne sont écrites nulle part ailleurs : c'est le serveur qui vous les donne.

2 Mener les cinq missions

Lancez votre agent sur les cinq missions, l'une après l'autre. Laissez-le travailler à sa façon : le but n'est pas de l'assister, c'est de **l'observer se tromper**.

Cette partie ne rapporte aucun point en elle-même — mais tout l'exercice 3 en sort. Une mission « réussie » trop vite est suspecte : vérifiez systématiquement le résultat contre les données brutes avant de le croire.

2.1 — Tenez un journal de bord

Pour chaque mission, notez au fil de l'eau : l'appel exact passé par l'agent, la réponse brute du serveur, ce que l'agent en a conclu, et ce qui clochait. Ce journal est votre matière première ; sans lui, l'exercice 3 se fait de mémoire et ça se voit.

a. Pour chacune des cinq missions : la réponse finale que vous retenez, et le nombre de tentatives qu'il a fallu.

b. Relevez au moins un cas où l'agent s'est déclaré satisfait d'un résultat **faux**. Capture de l'appel et de la réponse qui le prouvent.

> **⚠ CE SERVEUR N'EST PAS VOTRE AMI**
>
> Cette API est une vraie API d'entreprise : elle est vieille, incohérente, et elle ne vous préviendra pas quand elle vous ment. Un appel qui ne renvoie pas d'erreur n'est pas un appel qui a marché. Prenez l'habitude de lire la réponse brute, pas le résumé de l'agent.

3 Écrire le skill

C'est le cœur du TP. Vous produisez un **skill** qui documente les pièges de cette API. Critère de réussite : un agent qui ne connaît pas le serveur, mais qui dispose de votre skill, doit réussir les cinq missions **du premier coup**.

3.1 — Laissez l'agent l'écrire

Vous ne rédigez pas ce fichier à la main. C'est l'agent qui vient de se battre avec l'API : c'est lui qui sait ce qu'il a compris, et c'est à lui de l'écrire. Votre travail est de le diriger et de relire — un skill qui reste vague ou qui se trompe, c'est à vous de le renvoyer au travail.

Ne partez pas de zéro. Écrire un skill est un problème déjà résolu : il existe en ligne des commandes, des outils et des générateurs faits exactement pour ça, ainsi que la documentation officielle du format. Cherchez, installez, servez-vous-en — et dites dans votre rapport ce que vous avez trouvé et pourquoi vous l'avez choisi. Trouver l'outil existant fait partie de l'exercice ; le réécrire soi-même n'est pas valorisé.

3.2 — Ce que le skill doit contenir

Aucune structure n'est imposée : c'est le format attendu par OpenCode, et le reste vous regarde. En revanche le contenu doit tenir debout. Pour chaque piège, un lecteur doit pouvoir répondre à quatre questions : **quel outil** est concerné, **ce qu'on observe**, **ce que fait réellement le serveur**, et **la règle à appliquer** pour ne plus se faire avoir. Une entrée qui se contente de constater (« parfois ça renvoie vide ») sans cause ni parade ne vaut pas grand-chose.

Le nombre de pièges n'est pas communiqué : à vous de creuser jusqu'à en avoir fait le tour.

> **⚠ TOUT SIGNALER NE PAIE PAS**
>
> Un comportement signalé à tort comme une anomalie est compté contre vous. Certains outils ont un fonctionnement surprenant mais **parfaitement documenté dans leur propre description** : c'est du comportement normal, pas un piège. Lisez les descriptions des outils avant d'accuser.

TROUVER PLUS QUE PRÉVU, EN REVANCHE, ÇA PAIE

Cette API n'a pas été auditée ligne à ligne : **il est possible que vous tombiez sur un piège que personne n'avait relevé**. Mettez-le dans votre skill — une trouvaille inattendue est **valorisée**, jamais pénalisée, même si elle ne figurait sur aucune liste.

Ce qui départage les deux cas, c'est la **démonstration**, pas l'audace : l'appel exact et la **réponse brute** qui montre le problème, en quoi le serveur s'écarte de ce que sa propre description annonce, et la règle qui évite d'y retomber. Une intuition sans preuve reste un faux positif ; une preuve, non.

3.3 — La preuve

a. Repartez d'une session neuve, sans historique, avec le skill installé. Relancez les cinq missions. Captures avant / après sur au moins deux missions : le résultat sans le skill, le résultat avec.

b. Votre skill a-t-il été lu par l'agent ? Comment le savez-vous ? Capture du moment où OpenCode le charge.

c. Trois lignes : pourquoi ceci est-il un *skill* et pas une *command* ? Qu'est-ce qui change à l'usage ?

RENDU ATTENDU

- Un dépôt Git `tp1-<nom1>-<nom2>`, poussé, sans token en clair dans l'historique.
- Votre `opencode.json` (sans le token).
- Le skill, à l'emplacement attendu par OpenCode.
- Un `RAPPORT.md` : les réponses des exercices 1 à 3 et le journal de bord des cinq missions.
- Les **captures d'écran** demandées, dans un dossier du dépôt et référencées depuis le rapport. Les preuves sont des captures : un copier-coller de terminal ne compte pas.

MÉTHODE — TROIS RÉFLEXES QUI RAPPORTENT

- **Lisez les réponses brutes.** Le résumé de l'agent est une interprétation ; le piège est presque toujours dans le JSON.
- **Recoupez.** Deux chemins différents vers le même chiffre qui ne donnent pas le même chiffre : c'est là qu'il faut regarder.
- **Écrivez au fil de l'eau.** Le skill rédigé à la dernière demi-heure, de mémoire, se repère immédiatement à la correction.